

Large-scale Evaluation of Notebook Checkpointing with AI Agents

Hanxi Fang

University of Illinois Urbana-Champaign
Urbana, Illinois, USA
hanxif2@illinois.edu

Hari Sundaram

University of Illinois Urbana-Champaign
Urbana, Illinois, USA
hs1@illinois.edu

Supawit Chockchowwat

University of Illinois Urbana-Champaign
Urbana, Illinois, USA
supawit2@illinois.edu

Yongjoo Park

University of Illinois Urbana-Champaign
Urbana, Illinois, USA
yongjoo@illinois.edu

Abstract

Saving, or checkpointing, intermediate results during interactive data exploration can potentially boost user productivity. However, existing studies on this topic are limited, as they primarily rely on small-scale experiments with human participants—a fundamental constraint of human subject studies. To address this limitation, we employ AI agents to simulate a large number of complex data exploration scenarios, including revisiting past states and branching into new exploration paths. This strategy enables us to accurately assess the impact of checkpointing while closely mimicking the behavior of real-world data practitioners. Our evaluation results, involving more than 1,000 exploration paths and 2,848 executed code blocks, show that a checkpointing framework for computational notebooks can indeed enhance productivity by minimizing unnecessary code re-executions and redundant variables/code.

CCS Concepts

• **Human-centered computing** → Usability testing; User studies; Interactive systems and tools; • **Information systems** → Version management.

Keywords

AI agent-based evaluation, computational notebooks, version control systems, notebook checkpointing, interactive data science

ACM Reference Format:

Hanxi Fang, Supawit Chockchowwat, Hari Sundaram, and Yongjoo Park. 2025. Large-scale Evaluation of Notebook Checkpointing with AI Agents. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '25)*, April 26–May 01, 2025, Yokohama, Japan. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3706599.3719928>

1 Introduction

Checkpointing computational notebooks can improve user productivity [5]. Specifically, notebook systems like Jupyter [14], Colab [8],

R Markdown [3] let data scientists run a code block one at a time for analyzing tabular data, training machine learning models, visualizing results, etc. By checkpointing intermediate code/data, users can undo undesirable executions, explore alternative hypotheses, and restore from crashes. A recent work [5] proposes a checkpointing interface for computational notebooks and demonstrates its productivity benefits.

Unfortunately, the existing evaluation is limited to employing a handful of human subjects working on a fixed set of tasks. While those participants are allowed to freely explore, the study remains constrained by the number of participants, the variety of tasks, and the duration of the study. We recognize that these limitations are inherent to human subject studies: the limitations are difficult to overcome without adopting an entirely different approach.

In this work, we tackle them by devising an AI agent-based strategy. Our observation is that AI agents can generate high-quality code akin to that produced by real data scientists, and exhibit iterative refinement behaviors that closely resemble the testing, debugging, and code improvement processes typically employed by human practitioners. Specifically, we employ a pre-trained AI agent (i.e., ChatGPT 4o) to simulate real-world data scientists who explore data **with and without** a notebook checkpointing tool. For each of the scenarios, the only independent variable is the assistance offered by the checkpointing tool; however, the actual variables residing in sessions and also elapsed times vary significantly.

This “late-breaking work”, as a sequel to the recent research paper [5], expands the current state-of-the-art in two significant aspects. First, we systematically evaluate the effectiveness of the new code+data space checkpointing framework with hundreds of data exploration scenarios. This work is the first that evaluates data science checkpointing frameworks [20, 21, 24, 30] in such a large scale. Second, we reason why our agent-based approach is valid, ensuring two types of consistencies: consistency of generated code and consistency of branching strategy.

2 Background

We overview notebook checkpointing frameworks (Section 2.1) and discuss the limitations of the existing evaluation (Section 2.2).

2.1 Notebook Checkpointing

Computational notebooks (e.g., Jupyter, Colab) are designed around a linear, cell-by-cell execution model, even though real-world data

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CHI EA '25, Yokohama, Japan

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1395-8/25/04

<https://doi.org/10.1145/3706599.3719928>

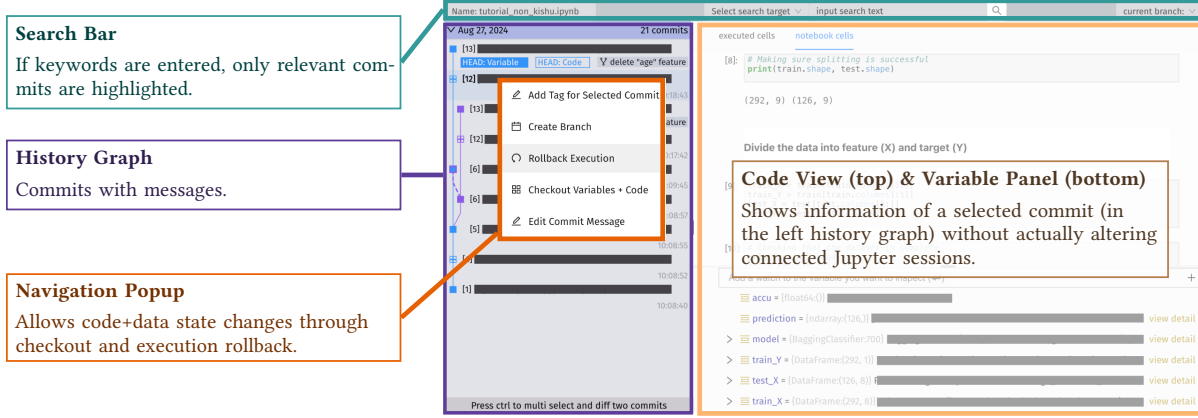


Figure 1: Background about Kishuboard. The history graph (purple box) shows past commits. The code and variable panes (yellow box) display the information of a selected commit. From any past commit, users can load data only (i.e., execution rollback) or load both code and data (i.e., checkout) using the navigation popup (red box). The image is reproduced with the original authors’ permission.

science exploration is often iterative and non-linear. Practitioners commonly revisit older states, branch the code to try different alternatives, and switch among branches. [17, 27] To accommodate branching, they either start a new notebook for each path [4] or mix branch-specific code cells in a single notebook [18, 22, 29]. Both approaches are time-consuming (re-running shared cells) and cognitively taxing (tracking which cells belong to each branch).

Inspired by code version control systems [23], Kishuboard [5] addresses these issues by introducing **two-dimensional code+data checkpointing** for computational notebooks (see Fig. 1 for its user interface). Unlike one-dimensional version control tools that track only code revisions, Kishuboard also version-controls the evolving “variable state in the kernel” or “data state” produced by each cell’s execution. Based on that, users can **checkout** the code and data state of notebook to any previous point and start exploring a new branch from that point. This feature, together with Kishuboard’s UI that visualizes the branched structure of exploration similar to Git GUI [7], enables fast, nonlinear data science exploration and reduces user’s cognitive burden of managing multiple exploration branches simultaneously.

2.2 Existing Evaluation of Notebook Checkpointing and its Limitations

The existing evaluation is based on human subject studies [5]. Specifically, to evaluate whether notebook checkpointing improves data science productivity, a user study was conducted with 20 student participants randomly split into experimental (with Kishuboard to checkout states) and control (without Kishuboard) groups. Both groups performed the same notebook-based tasks—building models, branching workflows, reporting metrics, retrieving previous variable states, debugging, and recovering from system crashes. While data practitioners may write their own code, participants in this study chose from a set of pre-written code snippets (with the option to modify them), ensuring that notebook checkpointing was the primary independent variable. The study

measured each participant’s task completion time and gathered feedback through surveys, then compared performance between the two groups.

Evaluation using traditional user study has several limitations:

- (1) As it relies on a small group of participants, its statistical power is limited.
- (2) The short study duration restricts participants’ ability to engage in extended exploration. Even if they are allowed to branch and iterate, the limited time reduces opportunities to test multiple hypotheses or recover from mistakes.
- (3) The task steps and reference code snippets are provided, limiting diverse exploration.

These limitations are inherent to resource-constrained human-subject user study. To address them under the user-study paradigm, one would need to recruit more participants, extend the study duration, and ensure uniform expertise in data science and Python coding—all of which may involve substantial costs.

3 AI-Agent-Based Experiment Design

We tackle the limitations mentioned in Section 2.2 using AI agents. As AI agents can generate diverse code in a short amount of time, compared to humans, it allows us to evaluate Kishuboard’s effectiveness at a much larger scale in terms of code diversity at a relatively small cost. We first describe our experiment design in this section. Then, we examine its scientific validity in Section 4.

Simulating Branched Data Exploration. We use a large language model (LLM), i.e., ChatGPT-4o [2], to iteratively generate code cells based on a generic task specification. We first provide the task: “My data is in file_name. The columns and their meanings are column_names_and_descriptions. I want to build a model to predict requirements.” then prompt the LLM to outline a step-by-step plan. Afterward, we prompt the agent to generate code cells and execute the generated code according to the plan step by step. Specifically, the execution results and code for previous steps are fed to the agent when generating the code for next step.

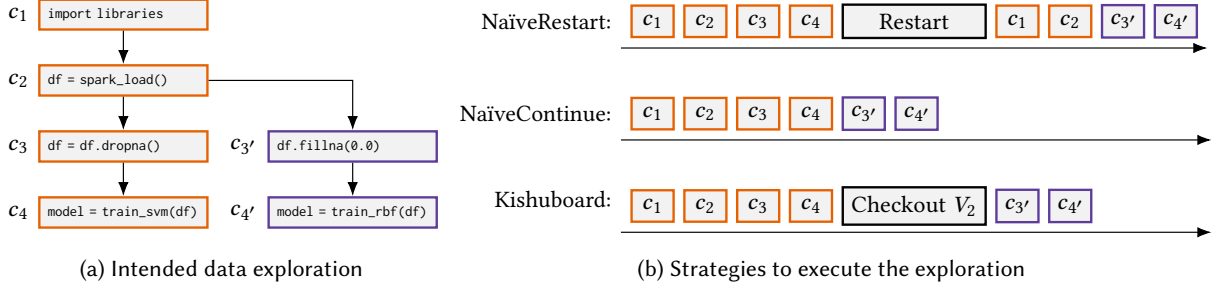


Figure 2: A toy example of user’s intended data exploration and strategies to execute it. NaïveRestart repetitively executes cells c_1 and c_2 . NaïveContinue executes new cells ($c_{3'}$ and $c_{4'}$) without any kernel restart. NaïveContinue may lead to branch interferences, for example, N/A values were already dropped by c_3 , making data imputation in $c_{3'}$ ineffective. Kishuboard restores checkpointed data to explore a new path, thus removing the repetitive work and preventing potential branch interferences.

Data scientists often revisit previous explored phases [9, 11, 18, 19, 22, 26]. To simulate such a behavior, we randomly select a past cell to revisit and ask the LLM to generate alternative code cells to continue from the selected cell. Each time creates a new branch of exploration. Whenever a runtime error occurs, the LLM can retry twice to self-correct according to the error message. If the LLM fails to fix the error, we checkout randomly and start a new branch. Appendix A presents concrete examples of our prompts and generated code by the LLM.

We repeat the generation process 10 times to simulate **10 exploration sessions** of data scientists exploring data independently where each session explores **100 branches**. The dataset is the 32 KB Titanic dataset downloaded from Kaggle [15]. Overall, the LLM generated **2,848** cells in this study.

To test Kishuboard’s performance with memory-intensive tasks, we also use the Spotify podcast dataset (i.e., 45 MB and 450 MB) [16] to simulate 3 exploration sessions with each exploring 3 branches.

Baselines and Kishuboard. Given an exploration session, we test the following existing baseline strategies and Kishuboard to support branched data exploration (see Figure 2 for an example). (1) **NaïveRestart**: To explore a new branch, NaïveRestart restarts the kernel and re-executes common cells to restore the selected past state. This strategy follows some user’s behaviour of starting a new notebook for a new exploration path [4]. (2) **NaïveContinue**: Instead, NaïveContinue appends new cells to the current notebook and continues execution from the current kernel state without any restart. It mimics a common practice to mix code from different cells within the same notebook [18, 22, 29]. NaïveContinue can introduce both explicit errors (e.g., operating on columns dropped in an earlier branch, thus observing execution errors) or implicit errors (e.g., unknowingly working on a dataframe updated by an earlier branch). (3) **Kishuboard**: This approach leverages Kishuboard to check out the code+data version to the latest common history state and continue execution from there.

Metrics. In addition to performance metrics (i.e., execution times), this experiment is also interested in measuring correctness and exploration complexity using the following metrics. (1) **End-to-end time to explore all branches**: This metric measures the time elapsed to execute and checkpoint/checkout code for all the

generated branches. It focuses solely on system efficiency during execution and state management (if any) as the time for AI-agents to generate code is neglected. (2) **Number of branch interferences**: In the real world, a *branch interference* occurs when a user reuses a modified variable assuming it was unmodified. Unfortunately, it may lead to misleading exploration outcomes which are hard to detect and debug. To study this type of mistake, we count the number of times a variable is incorrectly accessed across branches. For example, in Fig. 2, cell c_3 from the first branch drops rows with missing values in df , but with naively continue executing, cell $c_{3'}$ would ineffectively impute missing-but-already-dropped values which is a branch interference. We detect such branch interference by analyzing variable access and modification in each cell as well as manually verifying each detected branch interference. (3) **Peak number of variables**: This metric tracks the maximum number of variables in the kernel’s namespace. A higher number indicates a greater memory burden and increased cognitive overhead for the user to remember and manage the variables. (4) **Peak number of cells**: This metric tracks the maximum number of cells in the notebook at any given time. A larger number of cells indicates a messier notebook cell organization, leading to difficulties working with the notebook and cognitive burden.

4 Validation of the Experiment Design

In this section, we aim to justify the validity of AI-agent-based experiment design. Specifically, we discuss the similarity between AI agents and human behavior in terms of code generation. We also explain why the experiment setting effectively measures the efficiency gain of notebook checkpointing from a causal perspective.

Similarity to Human Exploration Behavior. Our decision to employ an AI agent for code generation and exploration is founded on recent research that demonstrates the capability of large language models (LLMs) to produce code (including data science code) and iterative refinement patterns that closely resemble those of human programmers [10, 13, 31, 32]. Concretely, LLMs has been shown to:

- (1) **Generate high-quality data science code**: LLMs can generate high-quality code for data science tasks. For example, recent research shows LLM-based agent can achieve accuracy of 94.9% on certain data analysis task benchmarks [13] and

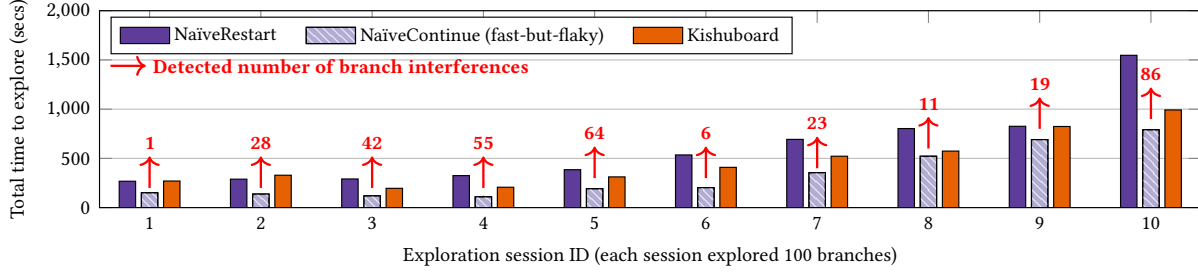


Figure 3: End-to-end execution time for Kishuboard and baseline methods. We generated 1000 branches of code using LLM-Agent, divided into 10 exploration sessions with 100 branches each. The sessions are sorted in ascending order by NaïveRestart time. NaïveContinue method is the fastest in terms of execution time, as it only runs newly added cells without checkpoint or checkout overhead. However, it is faulty, often producing incorrect results that do not trigger explicit errors, which may require significant debugging time. The additional time for the Kishuboard group is due entirely to checkpointing and checkout overhead, with the worst-case average overhead being just 2 seconds per branch. The red annotations indicate the number of implicit incorrect results for each session.

get mean ROC AUC performance of 0.82 on some feature engineering benchmarks [12].

- (2) **Iteratively refine code:** Research in LLM agents [28, 31, 33] shows that LLMs can parse feedback from external contexts to improve outputs. In our setup, the AI agent generates code step-by-step, receives execution feedback at each stage. Like a human data scientist, the AI agent can iteratively test, debug, and refine their work.

While the AI agent is not subject to the same cognitive constraints as a human (e.g., short-term memory limitations or domain biases), these overlaps in behavior patterns provide reasonable assurance that our experimental tasks and the corresponding branching strategies align with real-world exploration scenarios. Moreover, as many data scientists use AI for coding assistance [1, 6, 25], their code may employ part of AI-agent-generated code.

Causal Perspective on Measuring Efficiency Gains. A key rationale for our experiment is to measure, in a controlled manner, how much efficiency researchers can gain by using a notebook checkpointing mechanism (i.e., Kishuboard). From a causal inference perspective, the central challenge is to isolate the effect of the checkpointing strategy from other factors, such as task difficulty or participants’ skill level. Our design addresses this issue in two ways:

- (1) **Consistency of Generated Code:** By using the same generated code across different checkpointing strategies, we ensure that our experiment holds task complexity and baseline code constant. Because the generated code and data do not change, any observed differences in runtime behavior or outcomes must stem from the checkpointing mechanism itself.
- (2) **Consistency of Branching Strategy:** We introduce random selection to revisit previous cells, mirroring real-world data exploration where users backtrack to earlier steps [18, 22]. Each branch emerges from the same generative process but differs in terms of whether it restarts the kernel, continues with the existing kernel state, or checks out a prior code-data version. This design ensures that all strategies face the same exploration tasks, allowing a fair comparison of performance.

5 Experiment Result and Discussion

Using the AI-agent-based methodology described earlier, we present and discuss the experiment results in this section in terms of correctness, efficiency, and notebook simplicity. In summary, we get the following experiment results:

- (1) Kishuboard speeds up data exploration: up to 36% faster in both compute-intensive/light tasks, than naively restarting and re-execution. On average, data exploration with Kishuboard is 23% faster in compute-intensive tasks and 15% faster in compute-light tasks in terms of execution times.
- (2) Kishuboard prevents incorrect results that often occur when users naively explore alternative paths by appending cells and/or reusing variables.
- (3) Kishuboard maintains a clean notebook with only cells and variables in the current branch, which is significantly more effective than keeping track of branched structures with (almost) redundant code and variables.
- (4) Kishuboard’s space overhead—checkpoint sizes—is small.

Kishuboard Speeds up Data Exploration. We compare end-to-end times to explore using different strategies. Figure 3 shows that Kishuboard enables faster data explorations than NaïveRestart does. In Session 10, Kishuboard would save the user about 10 minutes of real-time or 36% faster than NaïveRestart. As the only exception, Kishuboard is slightly slower than NaïveRestart in Session 2 because cells before the branching point are all lightweight, making recomputation faster than using recovering from checkpoints. NaïveContinue is expectedly faster than Kishuboard as NaïveContinue overwrites the existing kernel state by running only the additional cells (for a new exploration path); however, NaïveContinue causes significant issues such as branch interferences and incorrect results as described below.

Kishuboard Ensures Correctness. Furthermore, Kishuboard guarantees correctness (Figure 3) by recovering exact data states from checkpoints. In contrast, while the alternative method NaïveContinue is the fastest, it is error-prone. In particular, we detect 355 branch interferences which are hard to detect and debug manually. For example, in Session 9, Branch 5 modifies a variable `X_test`

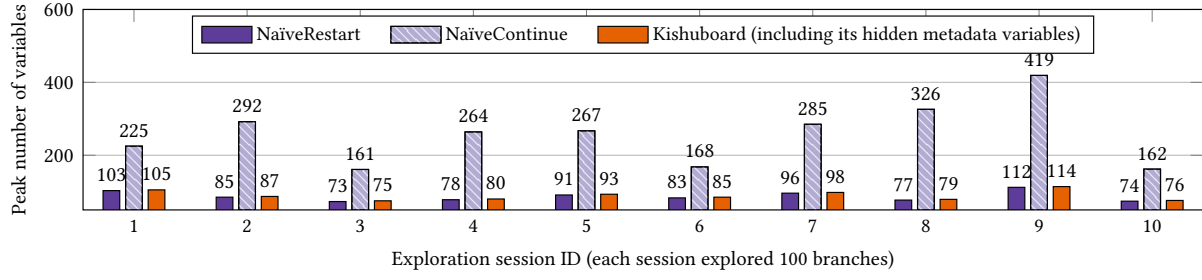


Figure 4: The peak number of kernel variables for each session during exploration. Smaller numbers of variables may be preferred for easier understanding. NaïveContinue produces excessive variables, increasing cognitive load to keep track of variables across branches. Kishuboard has exactly two more variables than NaïveRestart for user-invisible metadata.

by resampling from the original dataset and re-assigning it to the same variable name. Later on, Branch 7 evaluates its model based on `X_test` which leads to a misleading accuracy because the re-sampled `X_test` overlaps with the training data. Unsurprisingly, NaïveRestart also guarantees correctness but incurs computation costs to re-execute cells common across branches.

Kishuboard Maintains Clean Code and Data States. To assess notebook complexity, we measure the peak number of variables during exploration, as shown in Figure 4. Compared to NaïveRestart and Kishuboard, NaïveContinue results in significantly more variables, because NaïveContinue mixes data states from multiple exploration branches. We also measure the peak number of cells in the notebook during exploration and find that the NaïveContinue sessions have at least 24× more cells than the other two groups, as it appends code from different branches to the same notebook. Consequently, the user of NaïveContinue would experience an increased cognitive load to manage code cells across branches and remember variables.

Simulating Memory-intensive Tasks. Using the AI agent to explore both 45 MB and 450 MB Spotify dataset [16], our results presented in Table 1 show that Kishuboard can checkpoint variables efficiently without requiring extensive IO overhead or storage resources. For the 45 MB dataset, the total checkpoint size was 56 MB, and for the 450 MB dataset, it was 358 MB—smaller than the dataset itself in a CSV format. Furthermore, Kishuboard accelerates data exploration in both memory-light and memory-intensive scenarios, demonstrating the scalability of the method.

We observed that during experiments with larger datasets, Kishuboard occasionally fails with “`OverflowError: BLOB longer than INT_MAX bytes`” due to a limitation in the existing backend implementation [20]. Because this issue is an implementation flaw and not inherent to the technique, we believe that it could be addressed by chunking data before writing to storage.

6 Limitations of Experiment Setup and Result

While our AI-agent-based evaluation provides a large-scale and systematic analysis of notebook checkpointing, several limitations must be acknowledged. These primarily stem from the differences in behavior between AI agents and human practitioners.

AI Agents Do Not Replicate How Humans Iteratively Explore Branches. The simulation of 100 randomly generated branches

within a 10-cell notebook may not fully capture real-world data science workflows. While AI agents randomly revert to past commits, human users make such decisions based on logical reasoning or execution results. Furthermore, human users vary in exploration depth, leading to different numbers of executed cells per branch. This discrepancy may influence our findings, particularly in assessing the efficiency of different checkpointing strategies. For example, if humans always choose to checkout to a very early stage, then the efficiency gap between NaïveRestart and Kishuboard may be smaller.

Unlike AI Agents, Humans Can Mitigate Variable Overwrites. The drawbacks observed in the NaïveContinue group may not be as severe in real-world scenarios. Experienced users often mitigate variable overwrites by explicitly renaming or copying variables when switching between branches. As a result, the frequency of branch interferences leading to execution errors may be lower than what our AI-agent simulations suggest. Moreover, while AI agents append all executed cells into a single notebook, real users often refactor their workflow by removing obsolete cells or reorganizing their notebooks dynamically. However, it is important to note that these manual interventions take time—time that Kishuboard aims to save by automating refactoring and state management.

AI Agents Do Not Experience Cognitive Load. In real-world scenarios, users must actively navigate commit histories and decide which version to restore using Kishuboard. Similarly, those in the NaïveContinue and NaïveRestart groups would spend additional time figuring out how to organize cells or new notebooks. These interactions, which require human judgment and effort, are not fully accounted for in our AI-driven evaluation.

Given these factors, while our AI-agent-based approach offers valuable insights into checkpointing strategies at scale, complementary user studies are needed to validate its real-world applicability.

7 Conclusion

In this study, we demonstrate the efficacy of using AI agents to evaluate notebook checkpointing systems at scale, addressing limitations inherent in traditional human-subject studies. Traditional user studies are constrained by limited participant pools, short durations, and the substantial costs of ensuring uniform skill validation, which collectively hinder the scale and diversity of such evaluations. In contrast, AI agents offer a scalable alternative by generating diverse

Table 1: End-to-end execution time for exploring three randomly generated branches on small (45 MB) and large (450 MB) datasets. “Time” denotes the duration from executing the first cell of the first branch to completing the last cell of the final branch. “# of Cells” and “# of Variables” indicate the peak number of cells in the notebook and the peak number of variables in the kernel, respectively. Kishuboard outperforms NaïveRestart in efficiency for both dataset sizes, demonstrating the scalability of the method. Additionally, Kishuboard requires only 56 MB of storage for checkpoints on the small dataset and 358 MB on the large dataset throughout the exploration. While NaïveContinue is faster than Kishuboard, as observed in the previous experiments, NaïveContinue often causes hard-to-detect errors (Fig. 3) and creates significantly more variables (Fig. 4) and cells.

Method	Small Dataset (45 MB)			Large Dataset (450 MB)		
	Time (secs)	# of Cells	# of Variables	Time (secs)	# of Cells	# of Variables
NaïveRestart	38	9	63	271	9	63
NaïveContinue	18	14	69	141	14	69
Kishuboard	24	9	65	169	9	65

code in a short amount of time, mimicking real-world exploratory behaviors, and producing high-quality code. By simulating diverse and complex data exploration scenarios—encompassing a total of 1,000 branches—we assessed the performance of Kishuboard, a novel code+data checkpointing framework, against baselines. The results show that Kishuboard significantly enhances exploration efficiency, reduces cognitive burden, and prevents branching errors. Our work is the first of its kind to demonstrate the utility of AI agents in evaluating notebook checkpointing systems.

References

- [1] [n.d.]. GitHub Copilot. <https://github.com/features/copilot>. Accessed: 2025-01-23.
- [2] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [3] Benjamin Baumer and Dana Udwin. 2015. R markdown. *Wiley Interdisciplinary Reviews: Computational Statistics* 7, 3 (2015), 167–177.
- [4] Souti Chattopadhyay, Ishita Prasad, Austin Z Henley, Anita Sarma, and Titus Barik. 2020. What’s wrong with computational notebooks? Pain points, needs, and design opportunities. In *Proceedings of the 2020 CHI conference on human factors in computing systems*. 1–12.
- [5] Hanxi Fang, Supawit Chockchowwat, Hari Sundaram, and Yongjoo Park. 2025. Enhancing Computational Notebooks with Code+Data Space Versioning. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (conditionally accepted with minor revisions)*.
- [6] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. 2022. InCoder: A generative model for code infilling and synthesis. *arXiv preprint arXiv:2204.05999* (2022).
- [7] Git SCM. 2024. Git GUI Clients. <https://git-scm.com/downloads/guis> A list of graphical user interface (GUI) clients for Git.
- [8] Google. [n.d.]. Google Colab. <https://colab.research.google.com/>.
- [9] Phillip Guo. 2013. *Data Science Workflow: Overview and Challenges*. <https://cacm.acm.org/blogcacm/data-science-workflow-overview-and-challenges/>
- [10] Siyuan Guo, Cheng Deng, Ying Wen, Hechang Chen, Yi Chang, and Jun Wang. 2024. DS-Agent: Automated Data Science by Empowering Large Language Models with Case-Based Reasoning. *arXiv preprint arXiv:2402.17453* (2024).
- [11] Fred Hohman, Kanit Wongsuphasawat, Mary Beth Kery, and Kayur Patel. 2020. Understanding and visualizing data iteration in machine learning. In *Proceedings of the 2020 CHI conference on human factors in computing systems*. 1–13.
- [12] Noah Hollmann, Samuel Müller, and Frank Hutter. 2024. Large language models for automated data science: Introducing caafe for context-aware automated feature engineering. *Advances in Neural Information Processing Systems* 36 (2024).
- [13] Sirui Hong, Yizhang Lin, Bang Liu, Bangbang Liu, Binhao Wu, Ceyao Zhang, Chenxing Wei, Danyang Li, Jiaqi Chen, Jiayi Zhang, et al. 2024. Data interpreter: An llm agent for data science. *arXiv preprint arXiv:2402.18679* (2024).
- [14] Project Jupyter. 2023. Jupyter Notebook. <https://jupyter.org/>.
- [15] Kaggle. [n.d.]. Titanic - Machine Learning from Disaster. <https://www.kaggle.com/competitions/titanic>.
- [16] Kaggle. [n.d.]. Top Spotify Podcast Episodes (Daily Updated). <https://www.kaggle.com/datasets/daniilmlheev/top-spotify-podcasts-daily-updated>.
- [17] Mary Beth Kery, Bonnie E John, Patrick O’Flaherty, Amber Horvath, and Brad A Myers. 2019. Towards effective foraging by data scientists to find past analysis choices. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. 1–13.
- [18] Mary Beth Kery, Marissa Radensky, Mahima Arya, Bonnie E John, and Brad A Myers. 2018. The story in the notebook: Exploratory data science using a literate programming tool. In *Proceedings of the 2018 CHI conference on human factors in computing systems*. 1–11.
- [19] David Koop and Jay Patel. 2017. Dataflow notebooks: encoding and tracking dependencies of cells. In *9th USENIX Workshop on the Theory and Practice of Provenance (TaPP 2017)*.
- [20] Zhaoheng Li, Supawit Chockchowwat, Ribhav Sahu, Areet Sheth, and Yongjoo Park. 2024. Kishu: Time-Traveling for Computational Notebooks. *arXiv:2406.13856 [cs.DB]* <https://arxiv.org/abs/2406.13856>
- [21] Zhaoheng Li, Pranav Gor, Rahul Prabhu, Hui Yu, Yuzhou Mao, and Yongjoo Park. 2023. ElasticNotebook: Enabling Live Migration for Computational Notebooks. *arXiv preprint arXiv:2309.11083* (2023).
- [22] Jiali Liu, Nadia Boukhelifa, and James R Eagan. 2019. Understanding the role of alternatives in data analysis practices. *IEEE transactions on visualization and computer graphics* 26, 1 (2019), 66–76.
- [23] Jon Loeliger and Matthew McCullough. 2012. *Version Control with Git: Powerful tools and techniques for collaborative software development*. O’Reilly Media, Inc.
- [24] Naga Nithin Manne, Shilvi Satpati, Tanu Malik, Amitabha Bagchi, Ashish Gehani, and Amitabh Chaudhary. 2022. CHEX: multiversion replay with ordered checkpoints. *Proc. VLDB Endow.* 15, 6 (feb 2022), 1297–1310. <https://doi.org/10.14778/3514061.3514075>
- [25] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474* (2022).
- [26] Deepthi Raghunandan, Aayushi Roy, Shenzhi Shi, Niklas Elmqvist, and Leilani Battle. 2023. Code code evolution: Understanding how people change data science notebooks over time. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–12.
- [27] Adam Rule, Aurélien Tabard, and James D Hollan. 2018. Exploration and explanation in computational notebooks. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*. 1–12.
- [28] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2024. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems* 36 (2024).
- [29] Krishna Subramanian, Ilya Zubarev, Simon Völker, and Jan Borchers. 2019. Supporting data workers to perform exploratory programming. In *Extended Abstracts of the 2019 CHI Conference on Human Factors in Computing Systems*. 1–6.
- [30] April Yi Wang, Will Epperson, Robert A DeLine, and Steven M Drucker. 2022. Diff in the loop: Supporting data comparison in exploratory data analysis. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*. 1–10.
- [31] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [32] Frank F Xu, Uri Alon, Graham Neubig, and Vincent Josua Hellendoorn. 2022. A systematic evaluation of large language models of code. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*. 1–10.
- [33] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafraan, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629* (2022).

A AI-Agent Prompts and Generated Notebooks

A.1 Prompts

We first prompt ChatGPT-4 with the following prompts to generate steps it is planing to take, here's a real example:

"My dataset is in top_podcasts.csv, it has header. It has the following fields:<all fields name in the dataset>. I want to build a model to track how podcast rankings change over time to identify emerging trends.

It will be an interactive data science process using notebook. Tell me the steps to do this (without code). The steps should include some computation intensive ones such as hyperparameter tuning, model selection, neural network training, etc.

The step should be listed as:
There are XXX steps in total.
1. XXX
2. XXX

Next, we'll ask ChatGPT-4 to generate code step by step, along with the result of the last step feeded, here's an example:

"The output for this code cell is: <execution output>. Please generate the code for next step <step number>.

When the agent's output code contains error when executing, we give it 2 chances to self-correct, with the following prompt:

The output of the given code is: {output}, please help me debug it by generating the correct cell code.

Note that, when interacting with ChatGPT, we also include the history messages in the current prompt. When checking out, we also checkout the messages to the right point, so that ChatGPT only has the information about previous steps before the current step.

A.2 Example Generated Code

Here's a real example of the generated code branch, with step generated one by one as separate cells:

Step 1: Data Loading

```
import pandas as pd

# Load the dataset
file_path = 'titanic.csv'
titanic_data = pd.read_csv(file_path)

# Display the first few rows of the dataset
titanic_data.head()
```

Step 2: Data Exploration

```
import matplotlib.pyplot as plt
import seaborn as sns

# Display basic information about the dataset
titanic_data.info()

# Check for missing values
missing_values = titanic_data.isnull().sum()
print("Missing values in each column:\n", missing_values)

# Visualize the distribution of the 'Survived' variable
sns.countplot(x='Survived', data=titanic_data)
plt.title('Survival Counts')
plt.xlabel('Survived (0 = No, 1 = Yes)')
plt.ylabel('Count')
plt.show()
```

```
# Visualize the distribution of 'Age' with respect to 'Survived'
plt.figure(figsize=(12, 6))
sns.histplot(data=titanic_data, x='Age', hue='Survived', multiple='stack', bins=30)
plt.title('Age Distribution by Survival')
plt.xlabel('Age')
plt.ylabel('Count')
plt.show()
```

```
# Visualize the relationship between 'Pclass' and 'Survived'
sns.countplot(x='Pclass', hue='Survived', data=titanic_data)
plt.title('Survival Counts by Passenger Class')
plt.xlabel('Passenger Class')
plt.ylabel('Count')
plt.show()
```

```
# Pairplot to visualize relationships between features
sns.pairplot(titanic_data, hue='Survived', diag_kind='kde')
plt.show()
```

Step 3: Data Preprocessing

```
# Make a copy of the original dataset
titanic_data = titanic_data.copy()

# Handle missing values for 'Age' by filling with the median age
titanic_data['Age'].fillna(titanic_data['Age'].median(), inplace=True)

# Check again for missing values to confirm they have been addressed
missing_values_after = titanic_data.isnull().sum()
print("Missing values after preprocessing:\n", missing_values_after)

# Encode categorical variables using one-hot encoding for 'Embarked' and label encoding for 'Sex'
titanic_data = pd.get_dummies(titanic_data, columns=['Embarked'], drop_first=True)
titanic_data['Sex'] = titanic_data['Sex'].map({'male': 0, 'female': 1}) # Label encoding

# Drop columns that are not needed for model training
titanic_data.drop(['PassengerId', 'Name', 'Ticket'], axis=1, inplace=True)

# Display the preprocessed dataset
titanic_data.head()
```

Step 4: Feature Selection

```
# Import necessary libraries
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import SelectFromModel

# Define the features and target variable
X = titanic_data.drop('Survived', axis=1)
y = titanic_data['Survived']

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train a Random Forest Classifier to evaluate feature importance
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)

# Get feature importances
importances = rf_model.feature_importances_

# Create a DataFrame for feature importances
feature_importance_df = pd.DataFrame({'Feature': X.columns, 'Importance': importances})
feature_importance_df = feature_importance_df.sort_values(by='Importance', ascending=False)

# Display feature importances
```

```

print("Feature Importances:\n", feature_importance_df)

# Select features that are important (e.g., importance >= 0.05)
selected_features = feature_importance_df[feature_importance_df['
Importance'] >= 0.05]
['Feature'].tolist()
print("Selected Features:\n", selected_features)

# Create a new dataset with selected features
X_train_selected = X_train[selected_features]
X_test_selected = X_test[selected_features]

# Step 5: Data Splitting

# As we have already split the data in the previous step,
# we'll confirm the split and prepare for modeling.

# Display the shape of the training and testing sets
print("Training set shape (features):", X_train_selected.shape)
print("Testing set shape (features):", X_test_selected.shape)
print("Training set shape (target):", y_train.shape)
print("Testing set shape (target):", y_test.shape)

# Optionally, you may want to reset the indices of the training
# and testing sets for consistency
X_train_selected.reset_index(drop=True, inplace=True)
X_test_selected.reset_index(drop=True, inplace=True)
y_train.reset_index(drop=True, inplace=True)
y_test.reset_index(drop=True, inplace=True)

# Display the first few rows of the training set
print("First few rows of the training set (features):\n",
X_train_selected.head())
print("First few rows of the training set (target):\n", y_train.head
())

# Step 6: Model Selection

# Import necessary libraries
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

# Define a list of models to evaluate
models = {
    'Logistic Regression': LogisticRegression(),
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'Random Forest': RandomForestClassifier(random_state=42),
    'Support Vector Machine': SVC(random_state=42),
    'K-Nearest Neighbors': KNeighborsClassifier()
}

# Initialize a dictionary to hold model performance
model_performance = {}

# Fit each model and evaluate its performance
for model_name, model in models.items():
    model.fit(X_train_selected, y_train) # Fit the model
    y_pred = model.predict(X_test_selected) # Make predictions

    # Calculate accuracy
    accuracy = accuracy_score(y_test, y_pred)
    model_performance[model_name] = accuracy

    # Print classification report and confusion matrix for each model
    print(f"Model: {model_name}")
    print(f"Accuracy: {accuracy:.4f}")
    print("Classification Report:\n", classification_report(y_test,
y_pred))
    print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
    print("-" * 50)

# Display performance summary for all models

```

```

print("Model Performance Summary:\n", model_performance)

# Step 7: Hyperparameter Tuning

# Import necessary libraries for hyperparameter tuning
from sklearn.model_selection import GridSearchCV

# Define the model to tune (example: Random Forest)
model_to_tune = RandomForestClassifier(random_state=42)

# Define the hyperparameters to test
param_grid = {
    'n_estimators': [50, 100, 200], # Number of trees in the forest
    'max_depth': [None, 10, 20, 30], # Maximum depth of the tree
    'min_samples_split': [2, 5, 10], # Minimum number of samples
required to
                                #split an internal node
    'min_samples_leaf': [1, 2, 4] # Minimum number of samples
required to be at a leaf node
}

# Initialize GridSearchCV
grid_search = GridSearchCV(estimator=model_to_tune, param_grid=
param_grid,
                                scoring='accuracy', cv=5, verbose=1, n_jobs
=-1)

# Fit the model to the training data
grid_search.fit(X_train_selected, y_train)

# Get the best parameters and best score
best_params = grid_search.best_params_
best_score = grid_search.best_score_

print("Best Parameters from Grid Search:", best_params)
print("Best Cross-Validated Accuracy:", best_score)

# Optionally, we can also evaluate the best model on the test set
best_model = grid_search.best_estimator_
y_pred_best = best_model.predict(X_test_selected)

# Evaluate the best model on the test set
best_accuracy = accuracy_score(y_test, y_pred_best)
print(f"Best Model Test Accuracy: {best_accuracy:.4f}")
print("Classification Report of Best Model:\n",
classification_report(y_test, y_pred_best))
print("Confusion Matrix of Best Model:\n", confusion_matrix(y_test,
y_pred_best))

# Step 8: Model Training

# As we have already identified the best model through
hyperparameter tuning,
# we will now re-train this best model on the entire training
dataset.

# Fit the best model on the entire training set
best_model.fit(X_train_selected, y_train)

# Make predictions on the test set
y_pred_final = best_model.predict(X_test_selected)

# Evaluate the final model's performance on the test set
final_accuracy = accuracy_score(y_test, y_pred_final)
print(f"Final Model Test Accuracy: {final_accuracy:.4f}")
print("Final Classification Report:\n", classification_report(y_test,
y_pred_final))
print("Final Confusion Matrix:\n", confusion_matrix(y_test,
y_pred_final))

# Optionally, save the model for future use
import joblib
joblib.dump(best_model, 'best_random_forest_model.pkl')
print("Best model saved as 'best_random_forest_model.pkl'.")

```